OpenEye

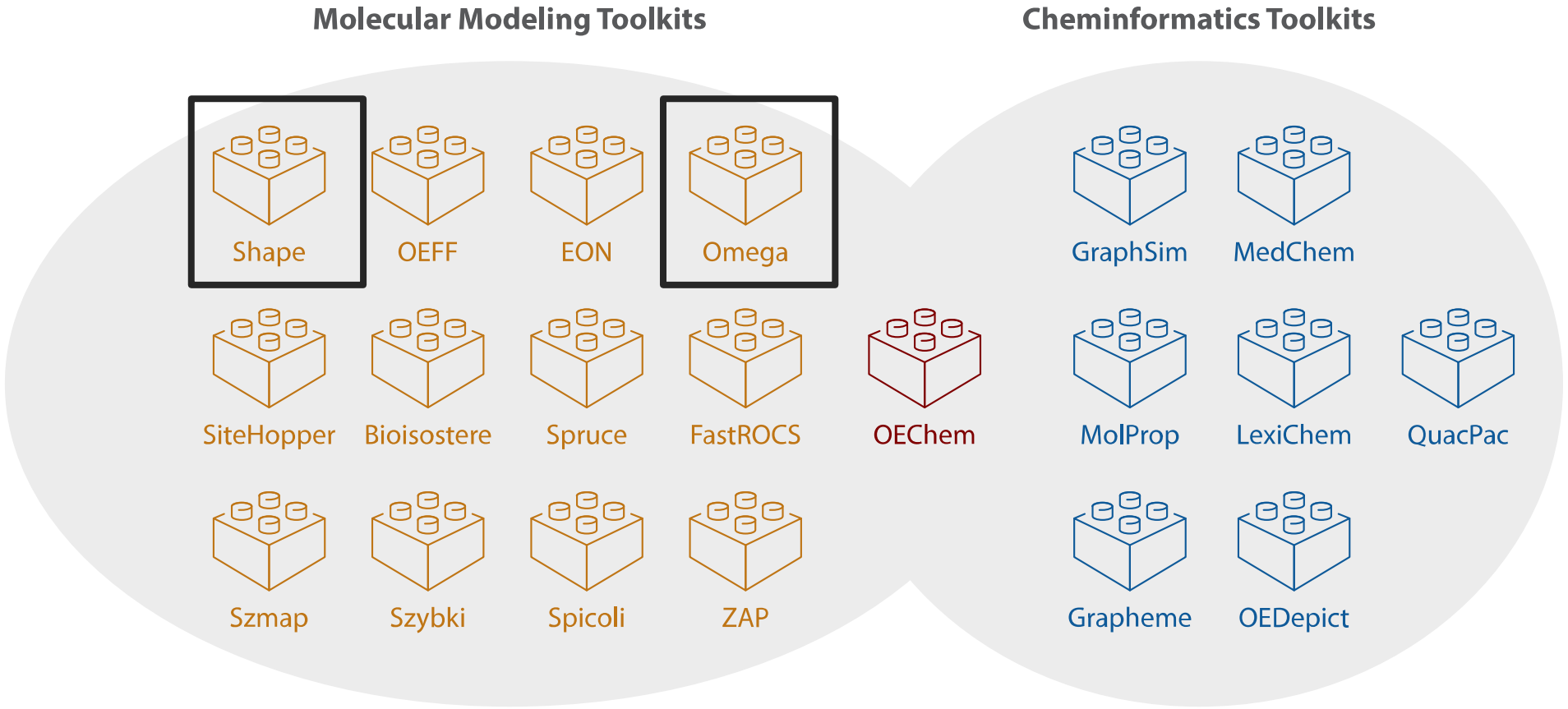
IS NOW A PART OF

cadence®

An OEMoleculeConformer Agent

Subscribe
& Follow

We provide programming libraries that *tackle* drug discovery problems



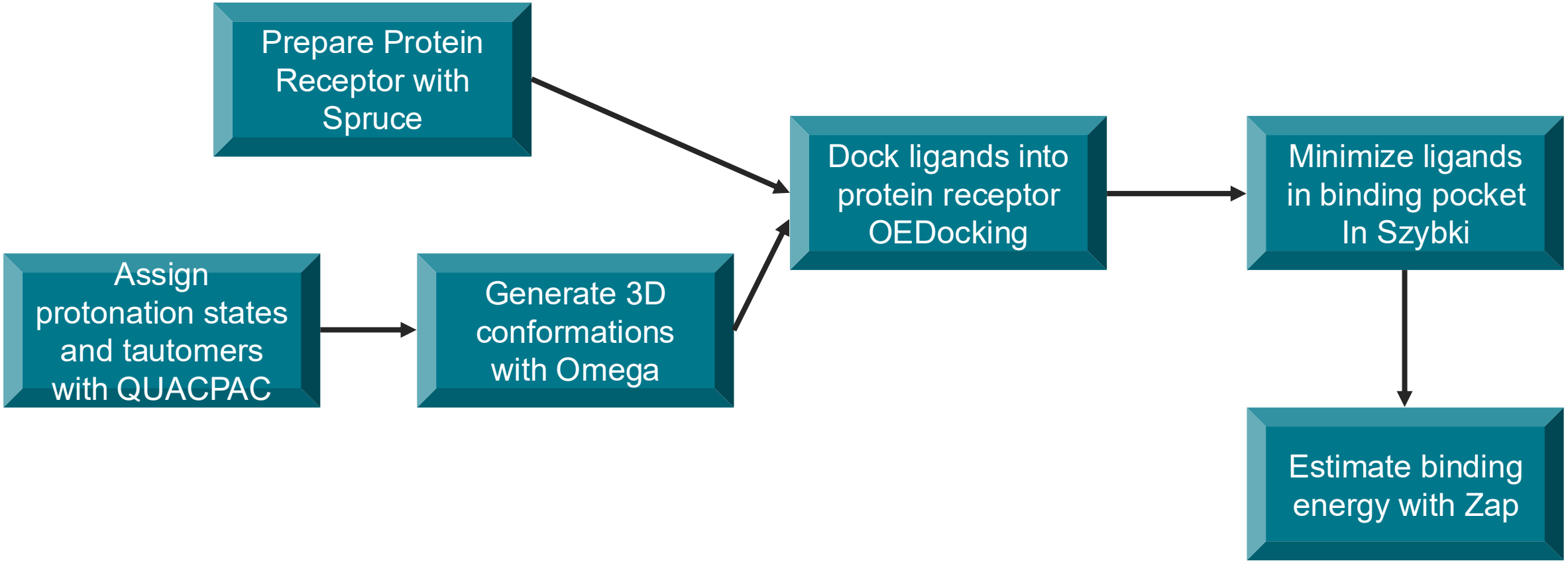
Libraries available in C++, C#, Java, and Python

OpenEye | cadence MOLECULAR SCIENCES

What are typical drug discovery problems?

- How can I *prepare* my protein structure?
- **How can I generate a conformational ensemble for my molecule?**
- What is the ionization state of my molecule?
- What is the likely binding pose of my molecule?
- **How similar is my molecule to other molecules in 3D and 2D space?**

What is a typical computational drug discovery workflow?



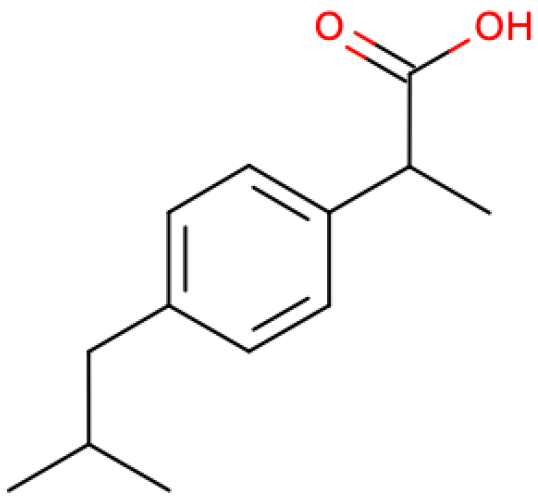
Now first all, what is a drug?

This is the formulation of a drug

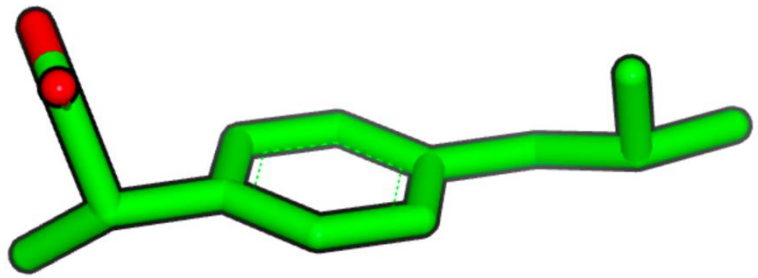
*A drug is a **molecule** that elicits a biological response often by interaction with a target to treat a disease by a safe and effective mechanism*

What is a molecule?

This is not a molecule



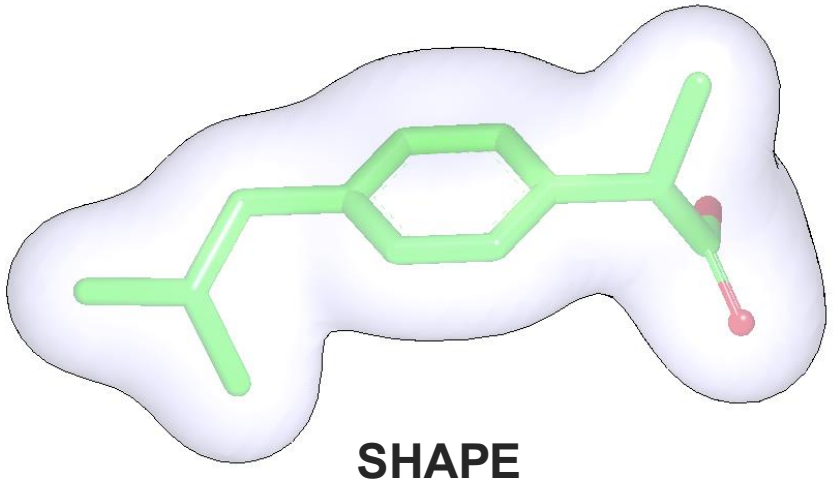
This is not a molecule



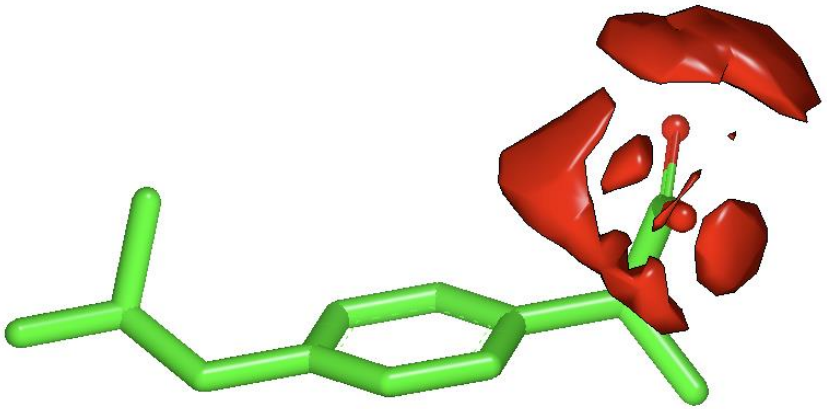
What *is* a molecule?

OpenEye | cadence MOLECULAR SCIENCES

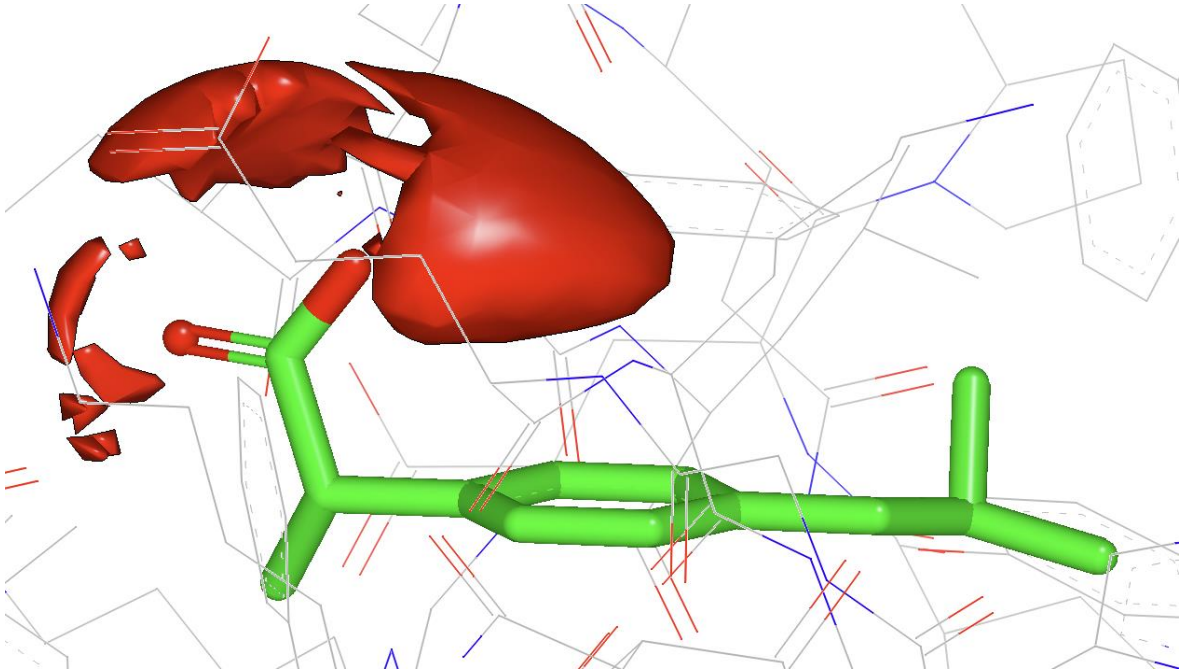
A molecule is based on 3D shape and electrostatics



SHAPE



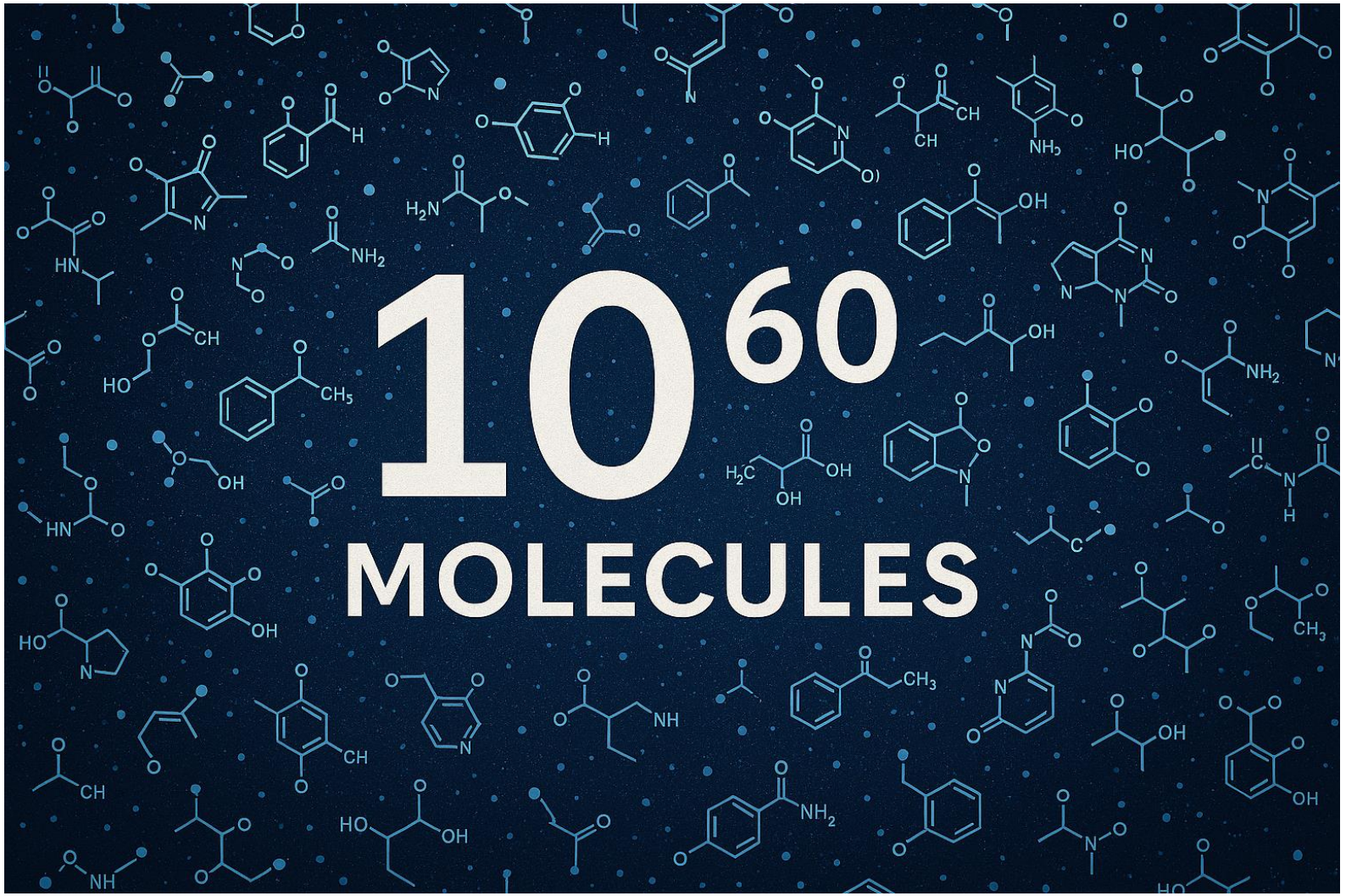
ELECTROSTATICS



That defines the interaction with a target

OpenEye | cadence MOLECULAR SCIENCES

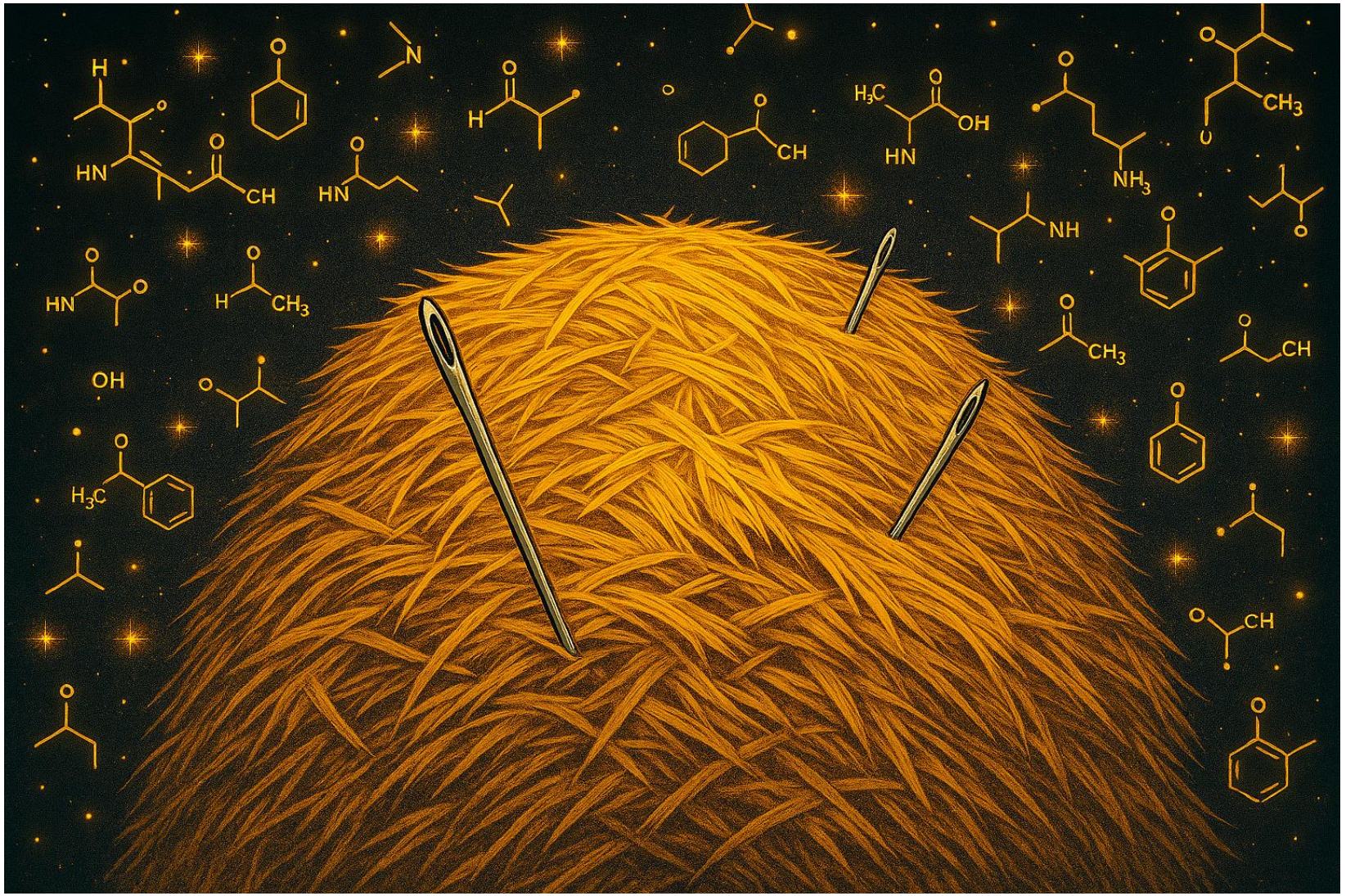
So how many molecules are there?



This is an AI-generated image

OpenEye | cadence MOLECULAR SCIENCES

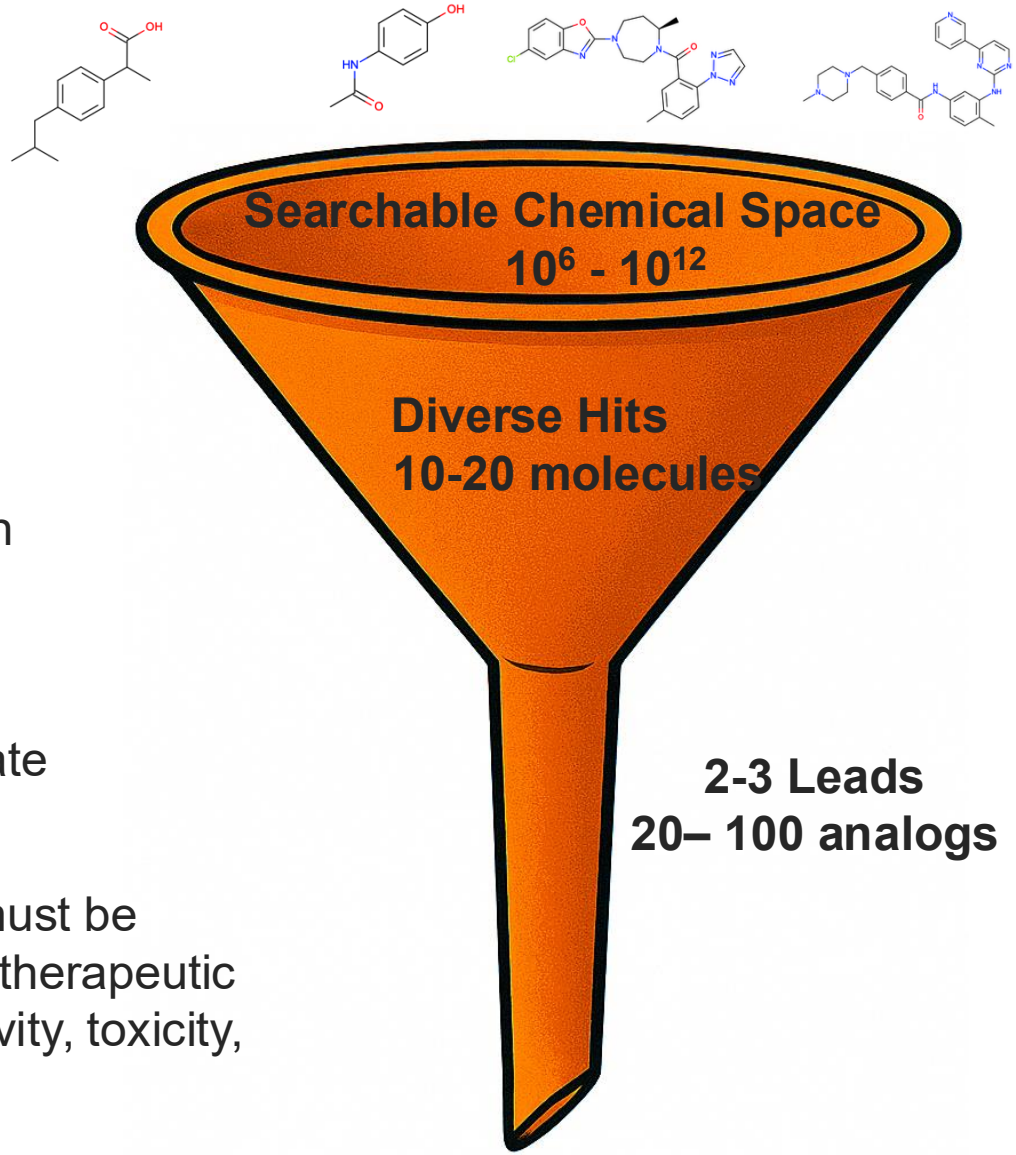
Computation helps to efficiently find the needles in the haystack



Extra credit: Can you spot the mistakes?

OpenEye | cadence MOLECULAR SCIENCES

The process of a Drug Discovery is a funnel from Hit to Drug



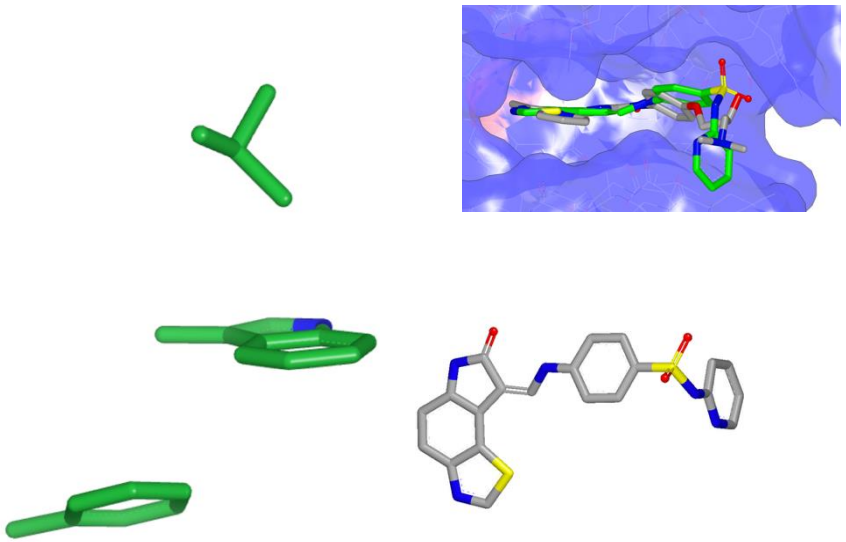
1) Target Identification
2) Hit Identification
3) Hit to Lead
4) Lead Optimization
5) Preclinical Candidate

For steps 3-5, molecules must be optimized to meet multiple therapeutic endpoints (potency, selectivity, toxicity, etc, etc)

Process takes 10-20 years and costs $500 million to $2.6 billion USD

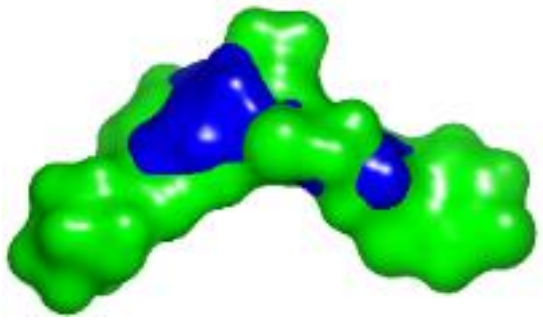
OpenEye | cadence MOLECULAR SCIENCES

ROCS helps to narrow down the funnel utilizing the **Shape Tk**



Single or Multiple Molecule Queries
(Disjointed, bioactive, or predicted conformer

Overlay



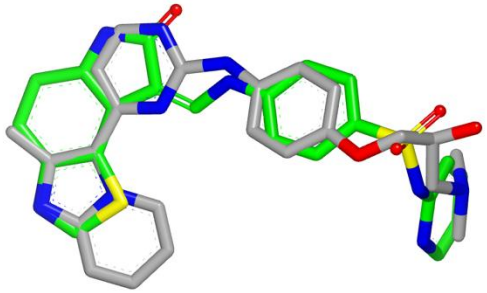
240 million conformers

eMolecules

100 billion conformers

Enamine

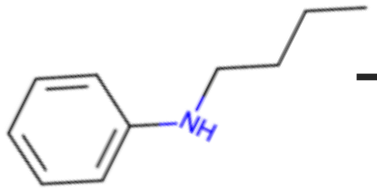
Score



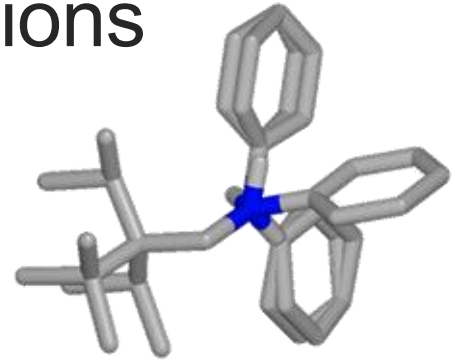
Shape Tanimoto = 0.77
Color Tanimoto = 0.23
TanimotoCombo = 1.00

OpenEye | cadence MOLECULAR SCIENCES

ROCS requires OMEGA to generate 3D conformations



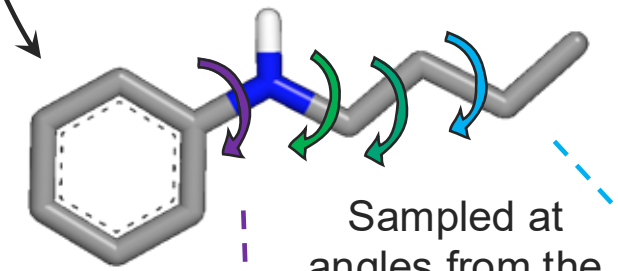
OMEGA



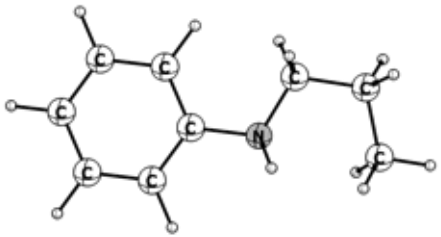
1) MolBuilder
Generate Initial Structure
(uses the Fragments Library)

2) Torsion Driving
Explores conformer space
(uses the Torsion Library)

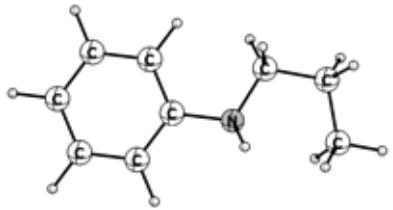
3) Ensemble Slicing
Selects conformers to keep in the ensemble.
(uses the hyperparameters like rms threshold, eWindow, MaxConfs)

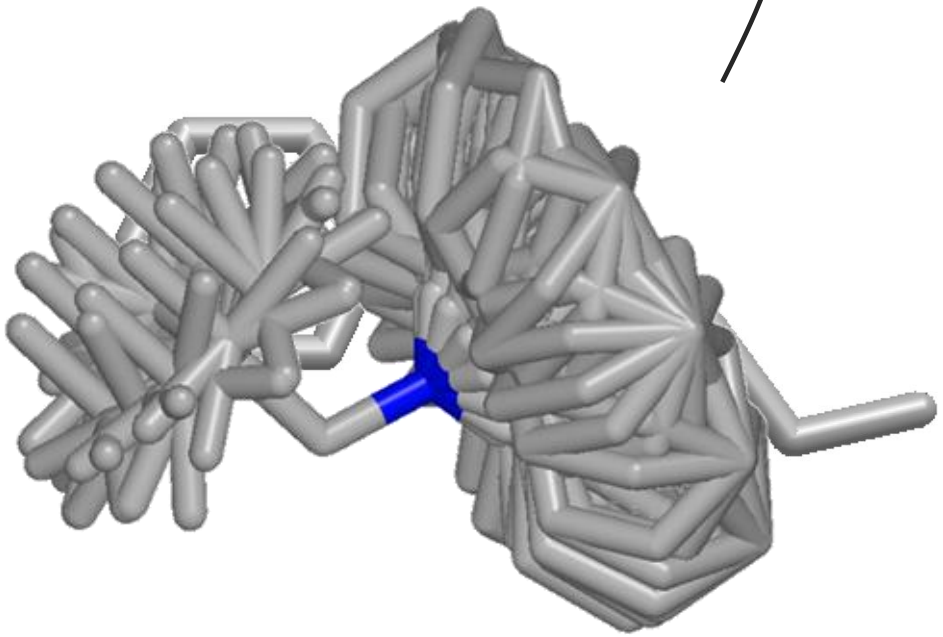


Sampled at angles from the Torsion Library



....





OpenEye | cadence MOLECULAR SCIENCES

The OpenEye API Architecture is made of 6 components

Base Classes

- *Chemistry-centric objects that facilitate all operations*
- OEMol(), OEAtom(), OEBond, OEConf(),
- OEGrid()
- OEMatch(), OEQMol()

Classes

- *Available modeling or cheminformatics methods*
- OEOmega()
- OEROCS()
- OEOverlay()
- OEHermite()
- OELibraryGen()

Option Classes

- *Accessories to Classes*
- *Low-level changes to available classes/methods*
- OEOmegaOptions()
- OEROCSOptions()
- OEOverlayOptions()
- OEUniMolecularRxnOptions()

Containers

- *Read-Only classes which may store results, molecules, fragments, interactions, etc*
- OEROCSResults()
- OEOverlapResults()
- OEBestOverlayScore()
- OEInteractionHintContainer()

Constants

- High-Level changes to available classes/methods
- OEOmegaSampling
- OETorLibType
- OEColorFFType

Functions

- *Calculations most often on molecules*
- OECalcPMI()
- OECalcVolume()
- OEFlipperStereoCenters()

OpenEye | cadence MOLECULAR SCIENCES

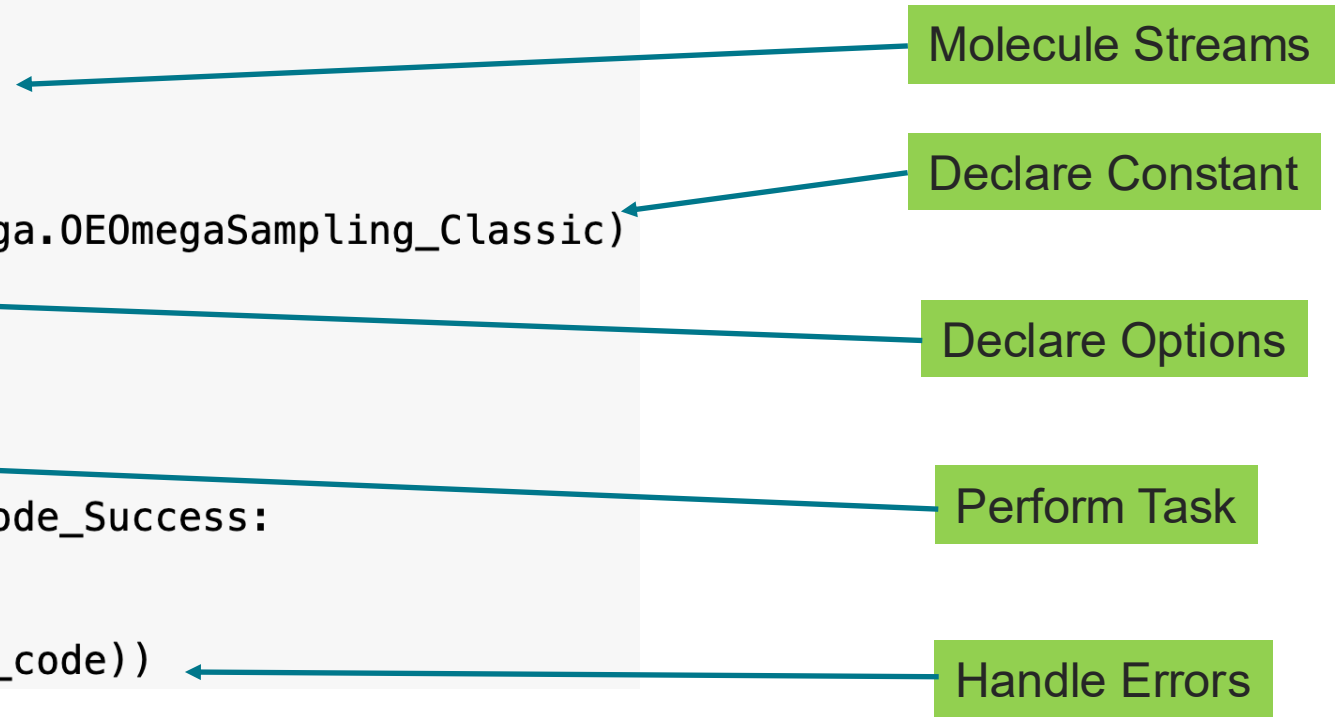
Implementation of Omega

```
from openeye import oeomega

ifs = oechem.oemolistream('input.smi.gz')
ofs = oechem.oemolostream('output.oez')

omega_opts = oeomega.OEOmegaOptions(oeomega.OEOmegaSampling_Classic)
omega_opts.SetMaxConfs(200)
omega = OEOmega(omega_opts)

for mol in ifs.GetOEMols():
    ret_code = omega.Build(mol)
    if ret_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(ret_code))
```



Improve productivity of engineers developing custom solutions to pharmaceutical challenges

- OpenEye's proprietary programming libraries are comprehensive, efficient, and allow engineers to create custom workflows and applications to solve problems in drug discovery

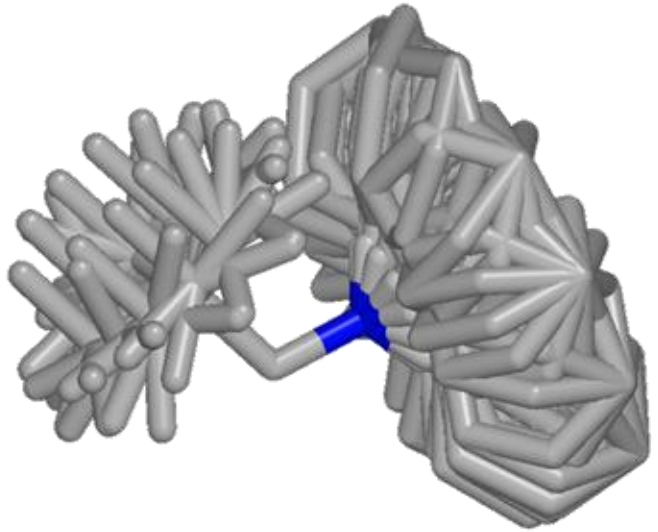
- However, the exhaustiveness of OpenEye's library can be a barrier for new engineers and hamper productivity

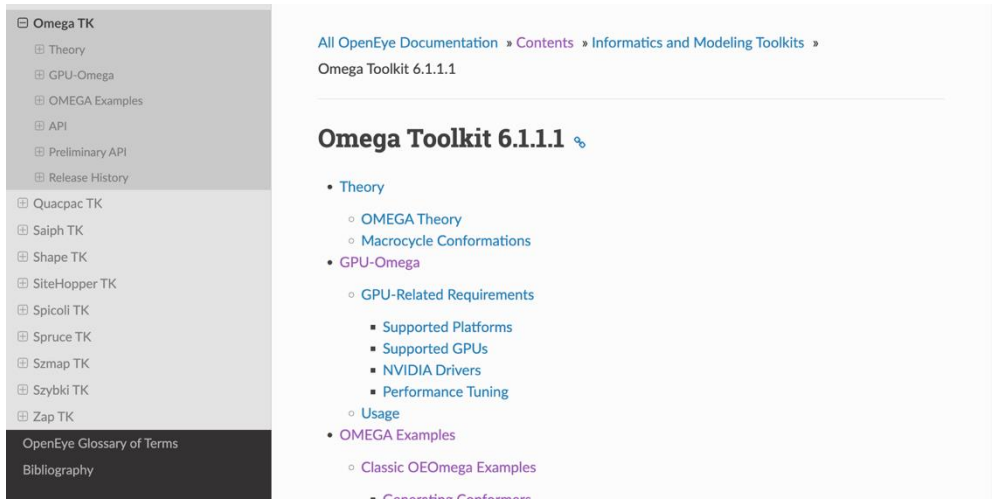
- The creation of a chatbot trained on OpenEye's documentation (https://docs.eyesopen.com/) with relevant code examples would help improve productivity and deliver solutions faster

OpenEye | cadence MOLECULAR SCIENCES

Given the timeframe, our goal will be to develop an initial chatbot trained on one OpenEye library (Omega)

- Docs can be found here:https://docs.eyesopen.com/toolkits/python/omegatk/index.html
- A PDF of the docs could also be provided
- Example prompts are available and shown on the remaining slides





Omega ChatBot Training Prompts

'Read a molecule from a file into OEChem'

```
from openeye import oechem
from sys import argv


ifs = oechem.oemolistream(argv[1])
mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)
```

'Write a molecule from OEChem'

```
from openeye import oechem
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

oechem.OEWriteMolecule(ofs,mol)
ofs.close()
```

"Generate conformations for my molecule with Omega"

```python
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate conformations for my database of molecules"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])


opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
omega = oeomega.OEOmega(opts)

for mol in ifs.GetOEMols():
    ret_code = omega.Build(mol)
    if ret_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Enumerate unspecified stereochemistry in my molecule with Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEFlipperOptions()

for isomer in oeomega.OEFlipper(mol,opts):
    oechem.OEWriteMolecule(ofs,mol)

ofs.close()
```

"Enumerate unspecified stereochemistry for my database of molecules with Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

opts = oeomega.OEFlipperOptions()

for mol in ifs.GetOEMols():
    for isomer in oeomega.OEFlipper(mol,opts):
        oechem.OEWriteMolecule(ofs,isomer)

ofs.close()
```

"Enumerate unspecified stereochemistry for molecules up to 3 max unspecified stereocenters"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

opts = oeomega.OEFlipperOptions()
opts.SetMaxCenters(3)

for mol in ifs.GetOEMols():
    for isomer in oeomega.OEFlipper(mol,opts):
        oechem.OEWriteMolecule(ofs,isomer)

ofs.close()
```

"Enumerate unspecified stereochemistry for molecules up to 5 max unspecified stereocenters"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

opts = oeomega.OEFlipperOptions()
opts.SetMaxCenters(5)

for mol in ifs.GetOEMols():
    for isomer in oeomega.OEFlipper(mol,opts):
        oechem.OEWriteMolecule(ofs,isomer)

ofs.close()
```

"Enumerate unspecified stereocenters and generate 3D conformers with Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

omega_opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
omega = oeomega.OEOmega(omega_opts)
opts = oeomega.OEFlipperOptions()

for isomer in oeomega.OEFlipper(mol,opts):
    imol = oechem.OEMol(isomer)
    ret_code = omega.Build(imol)
    if ret_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,imol)
    else:
        print(oeomega.OEGetOmegaError(ret_code)

ofs.close()
```

"Enumerate unspecified stereocenters and generate 3D conformers with Omega for a molecule database"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])


omega_opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
omega = oeomega.OEOmega(omega_opts)
opts = oeomega.OEFlipperOptions()

for mol in ifs.GetOEMols():
    for isomer in oeomega.OEFlipper(mol,opts):
        imol = oechem.OEMol(isomer)
        ret_code = omega.Build(imol)
        if ret_code == oeomega.OEOmegaReturnCode_Success:
            oechem.OEWriteMolecule(ofs,imol)
        else:
            print(oeomega.OEGetOmegaError(ret_code)

ofs.close()
```

"Generate 3D conformations with Omega with max confs of 1"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetMaxConfs(1)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations using with Omega for docking"

```python
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Pose)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations using Omega for ROCS"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_ROCS)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

'Generate 3D conformations with Omega for FastROCS"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_FastROCS)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations with Omega for freeform"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Dense)
omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations with GPU Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_drive_opts.SetUseGPU(True)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetTorDriveOptions(tor_drive_opts)

omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations with Omega using thompson sampling"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_drive_opts.SetUseThompson(True)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetTorDriveOptions(tor_drive_opts)

omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformastions with Omega using alternative torsion library"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_drive_opts.SetTorLib(oeomega.OETorLibType_GubaV21)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetTorDriveOptions(tor_drive_opts)

omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations with Omega using electrostatics"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_drive_opts.SetForceField('mmff94s_Sheff')

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetTorDriveOptions(tor_drive_opts)

omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations with a fixed substructure"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

ss = oechem.OESubSearch('<SMILES string>')
ss.SetMaxMatches(1)

conf_fix_options = oeomega.OEConfFixOptions()
conf_fix_options.SetFixSubSearch(ss)

mol_builder_opts = oeomega.OEMolBuilderOptions()
mol_builder = oeomega.OEMolBuilder(mol_builder_opts)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_driver = oeomega.OETorDriver(tor_drive_opts)


build_code = mol_builder.Build(mol,conf_fix_options)
if build_code == oeomega.OEOmegaReturnCode_Success:
    gen_code == tor_driver.GenerateConfs(mol,conf_fix_options
    if gen_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(gen_code))
else:
    print(oeomega.OEGetOmegaError(build_code))
```

"Generate 3D conformations with a fixed substructure given a SMARTS patttern"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

conf_fix_options = oeomega.OEConfFixOptions()
conf_fix_options.SetFixSmarts('<SMARTS String>')

mol_builder_opts = oeomega.OEMolBuilderOptions()
mol_builder = oeomega.OEMolBuilder(mol_builder_opts)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_driver = oeomega.OETorDriver(tor_drive_opts)


build_code = mol_builder.Build(mol,conf_fix_options)
if build_code == oeomega.OEOmegaReturnCode_Success:
    gen_code == tor_driver.GenerateConfs(mol,conf_fix_options)
    if gen_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(gen_code))
else:
    print(oeomega.OEGetOmegaError(build_code))
```


OpenEye | cadence MOLECULAR SCIENCES

"Generate 3D conformations of a molecule and hold a piece of the molecule fixed"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

fixed_fs = oechem.oemolistream(argv[3])
fixed_mol = oechem.OEMol()
oechem.OEReadMolecule(fixed_fs,fixed_mol)

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

conf_fix_options = oeomega.OEConfFixOptions()
conf_fix_options.SetFixMol(fixed_mol)

mol_builder_opts = oeomega.OEMolBuilderOptions()
mol_builder = oeomega.OEMolBuilder(mol_builder_opts)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_driver = oeomega.OETorDriver(tor_drive_opts)


build_code = mol_builder.Build(mol,conf_fix_options)
if build_code == oeomega.OEOmegaReturnCode_Success:
    gen_code == tor_driver.GenerateConfs(mol,conf_fix_options)
    if gen_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(gen_code))
else:
    print(oeomega.OEGetOmegaError(build_code))
```


OpenEye | cadence MOLECULAR SCIENCES

"Generate 3D conformations and apply MCS constraint on a portion of my molecule"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

fixed_fs = oechem.oemolistream(argv[3])
fixed_mol = oechem.OEMol()
oechem.OEReadMolecule(fixed_fs,fixed_mol)

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

conf_fix_options = oeomega.OEConfFixOptions()
conf_fix_options.SetFixMol(fixed_mol)
conf_fix_options.SetFixMCS(True)

mol_builder_opts = oeomega.OEMolBuilderOptions()
mol_builder = oeomega.OEMolBuilder(mol_builder_opts)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_driver = oeomega.OETorDriver(tor_drive_opts)


build_code = mol_builder.Build(mol,conf_fix_options)
if build_code == oeomega.OEOmegaReturnCode_Success:
    gen_code == tor_driver.GenerateConfs(mol,conf_fix_options)
    if gen_code == oeomega.OEOmegaReturnCode_Success:
        oechem.OEWriteMolecule(ofs,mol)
    else:
        print(oeomega.OEGetOmegaError(gen_code))
else:
    print(oeomega.OEGetOmegaError(build_code))
```


OpenEye | cadence MOLECULAR SCIENCES

"Generate 3D conformations of a macrocycle molecule with Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEMacrocycleOmegaOptions()
omega = oeomega.OEMacrocycleOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformations of a macrocycle molecule in a specific solution"

```python
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

# Set Dielectric Constant to change solution type. This is dielectric constant of water.
mc_builder_opts = oeomega.OEMacrocycleBuilderOptions()
mc_builder_opts.SetDielectricConst(80.0)

opts = oeomega.OEMacrocycleOmegaOptions()
opts.SetMacrocycleBuilderOptions(mc_builder_opts)

omega = oeomega.OEMacrocycleOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate 3D conformation of a macrocycle"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)


opts = oeomega.OEMacrocycleOmegaOptions()
opts.SetMaxConfs(1)

omega = oeomega.OEMacrocycleOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Calculate the number of stereocenters of a molecule with Omega"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

opts = oeomega.OEFlipperOptions()

num_centers = oeomega.OEFlipperStereoCenters(mol,opts)
```

"Check if molecule is a macrocycle"

[ ]:
```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

if oeomega.OEIsMacrocycle(mol):
    print('Yes, this molecule is a macrocycle')
else:
    print('No this is not a macrocycle')
```

"Remove duplicate conformers from ensemble"

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

slice_ensemble_opts = oeomega.OESliceEnsembleOptions()
conf_slicer = oeomega.OEConfSlicer(slice_ensemble_opts)

if conf_slicer.Slice(mol):
    oechem.OEWriteMolecule(ofs,mol)
```

'Write out conformer energy'

```
from openeye import oechem,oeomega
from sys import argv

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

mol = oechem.OEMol()
oechem.OEReadMolecule(ifs,mol)

tor_drive_opts = oeomega.OETorDriveOptions()
tor_drive_opts.SetSDEnergy(True)

opts = oeomega.OEOmegaOptions(oeoemga.OEOmegaSampling_Classic)
opts.SetTorDriveOptions(tor_drive_opts)

omega = oeomega.OEOmega(opts)
ret_code = omega.Build(mol)
if ret_code == oeomega.OEOmegaReturnCode_Success:
    oechem.OEWriteMolecule(ofs,mol)
else:
    print(oeomega.OEGetOmegaError(ret_code))

ofs.close()
```

"Generate Missing Fragments from input database with Omega"

```
from openeye import oechem,oeomega
from sys import argv

make_lib = oeomega.OEMakeFragLib()
make_lib.AddFragLib()

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

make_lib.GenerateMissingFrags(ifs,ofs)
```

"Retrieve Missing Fragments from fragment library in Omega"

```
from openeye import oechem,oeomega
from sys import argv

make_lib = oeomega.OEMakeFragLib()
make_lib.AddFragLib()

ifs = oechem.oemolistream(argv[1])
ofs = oechem.oemolostream(argv[2])

for mol in make_lib.GetMissingFrags(ifs):
    oechem.OEWriteMolecule(ofs,mol)

ofs.close()
```

Subscribe to our mailing list

OpenEye | cadence® MOLECULAR SCIENCES